代码说明 • 技术设计 • 复现验证

数值库技术说明

Eigen · GTest · Google Benchmark · pybind11

代码：numerics/

构建：CMake 3.18+, C++17

依赖：Eigen/GTest/Benchmark/pybind11

版本：v1.0 • 2026-11

> *本模块是通用数值核心，不包含望远镜业务；所有测试均可用解析解/NumPy/Eigen 对照。*

01 模块定位与 API

只做四类运算，每个都能独立验证。

表 1-1 模块结构

文件	职责	接口
numerics.hpp	公开 API + Doxygen 注释	solve_linear/least_squares/fft/solve_ode_*
numerics.cpp	Eigen 实现 + 异常检查	同上
tests/test_numerics.cpp	10 个 GTest	解析解/NumPy/Eigen 对照
bench/bench_numerics.cpp	4 组 Benchmark	solve/lstsq/FFT/RK4
bindings/numerics_py.cpp	pybind11	telescope_numerics_py

代码 1-1 公开接口。

```
Vector x = solve_linear(A, b);       // LU, partial pivot
Vector c = least_squares(A, b);      // QR
Vector p = fit_polynomial(xs, ys, 2); // polynomial LSQ
auto X = fft_copy(x);                // radix-2
OdeResult r = solve_ode_scalar(rhs, y0, t0, t1, dt);
```

02 数学原理

LU、QR/SVD、FFT、RK4 的假设/复杂度/失败模式。

2.1 线性方程组

部分主元 LU：P A = L U，解 L y = P b、U x = y；O(N³)。主元选绝对值最大行，抑制舍入放大。奇异/非方阵抛异常，不返回 NaN。

2.2 最小二乘

超定系统 min ||A x − b||₂。正规方程 AᵀA x = Aᵀb 会把条件数平方；本库默认列主元 Householder QR，并提供 SVD 版本处理秩亏/病态。复杂度 O(MN²)。

2.3 FFT

radix-2 Cooley–Tukey：X[k]=Σ x[n]exp(−2πi nk/N)，逆变换共轭/N；只接受 2 的幂，否则抛异常。复杂度 O(N log N)。

2.4 RK4

y' = f(t,y)，k1..k4 加权平均；局部 O(h⁵)，全局 O(h⁴)。标量返回 t/y，向量返回 t/states。固定步长，刚性问题应换隐式方法。

03 软件设计与代码注释

为什么这样写，注释写在哪。

表 3-1　设计决策

设计点	做法	原因
值语义/RAII	Eigen Matrix/Vector	无裸 new，异常安全
异常	invalid_argument/runtime_error	尺寸/奇异/非 2 的幂显式失败
PIC	POSITION_INDEPENDENT_CODE ON	pybind11 共享库可链接
依赖	系统包优先 + FetchContent	可复现；WSL 避免误链 Windows conda
注释	文件头 + 函数前 Doxygen	输入/输出/异常/单位

代码 3-1　文件头注释示例。

```
/// @file numerics.hpp
/// @brief Small, testable numerical core.
/// @throws std::invalid_argument wrong shape / non power-of-two FFT
/// @throws std::runtime_error singular matrix
```

3.2 构建与依赖

CMake 3.18+；Eigen/GTest/Google Benchmark/pybind11 优先 find_package，找不到用 FetchContent。WSL 下 find_package(GTest) 可能误链 Windows conda 的 gtest.lib，默认改为 FetchContent。静态库必须 POSITION_INDEPENDENT_CODE ON，否则 pybind11 共享模块链接报 relocation。

表 3-2　CMake 选项

选项	默认	作用
TEL_NUMERICS_BUILD_TESTS	ON	GTest 10 项
TEL_NUMERICS_BUILD_BENCH	ON	Google Benchmark
TEL_NUMERICS_BUILD_PYTHON	ON	pybind11 模块
TEL_NUMERICS_ENABLE_ASAN	OFF	AddressSanitizer
TEL_NUMERICS_USE_SYSTEM_GTEST	OFF	默认 FetchContent，避免跨平台 .lib

3.3 错误处理

表 3-3　异常契约

输入	行为	测试
A 非方阵	invalid_argument	RejectsSingularAndBadShape
A 奇异	runtime_error	RejectsSingularAndBadShape
FFT 非 2 的幂	invalid_argument	Fft.ImpulseAndRoundTrip
ODE dt<=0	invalid_argument	Ode.RejectsBadStep

04 测试

10 个 GTest，覆盖正常/边界/异常。

表 4-1　GTest（10/10）

测试	验证	判据
Known3x3	已知解	残差 <1e-12
RejectsSingular	奇异/坏形状	抛异常
LineFit	直线拟合	误差 <0.02
SvdMatchesQr	QR vs SVD	解差 <1e-10
FitQuadratic	二次多项式	系数 <1e-9
FftImpulse	冲激/逆变换	误差 <1e-12
FftNaive	朴素 DFT	误差 <1e-10
OdeExponential	y'=y	误差 <1e-7
OdeOscillator	谐振子	误差 <2e-6
OdeBadStep	dt=0	抛异常

代码 4-1　构建/测试/ASan。

```
cmake -S numerics -B build-numerics -DCMAKE_BUILD_TYPE=Release
cmake --build build-numerics -j4
ctest --test-dir build-numerics --output-on-failure
ctest --test-dir build-numerics-asan --output-on-failure
```

4.2 测试设计

测试不是“跑通”，而是三件事：① 解析解（冲激谱、指数、谐振子）；② 独立实现对照（朴素 DFT、NumPy/Eigen）；③ 失败路径（奇异、非方阵、非 2 的幂、负步长）。ASan 构建在 WSL 下用单个 CTest 入口，避免 gtest_discover 与 ASan 冲突。

4.3 回归

任何实现改动后，先跑 10 项 GTest；接口/数值变化必须更新测试或说明原因。CI 的 numerics job 会重复这套。

05 性能与 Python 绑定

复杂度与实测。

表 5-1 Benchmark（本机 CPU）

基准	规模	时间
solve_linear	8	~0.50 µs
solve_linear	32	~11.8 µs
solve_linear	128	~0.57 ms
least_squares	1024×64	~0.69 ms
FFT	1024	~13.8 µs
FFT	4096	~67 µs
RK4	1000 steps	~14 µs

代码 5-1 pybind11 调用。

```
import numpy as np, telescope_numerics_py as tn
print(tn.solve_linear(np.eye(3), np.ones(3)))
print(tn.fft([1+0j, 0j, 0j, 0j]))
```

5.2 Benchmark 方法

用 Google Benchmark 固定迭代/最小时间，避免单次计时噪声；solve 用 8/32/128 三个规模拟合 N^3，FFT 用 256/1024/4096 拟合 N log N。结果只作量级参考，受 CPU/编译器/缓存影响。

5.3 Python ABI

pybind11 模块必须和编译时 Python 版本一致。本机用 /usr/bin/python3 3.8；若 CMake 找到 uv 的 3.12，需要显式指定 pybind11 cmake dir 或 PYTHON_EXECUTABLE。

5.4 使用示例

代码 5-2 C++/Python 等价示例。

```
A = [[4,-2,1],[-2,4,-2],[1,-2,4]]; b=[11,-16,17]
x = solve_linear(A,b)          # [1,-2,3]
# polynomial least squares
c = fit_polynomial([0,1,2,3],[1,3,7,13],2)  # [1,1,1]
```

5.5 边界

- 不处理稀疏矩阵/迭代法；
- FFT 只支持 2 的幂；
- RK4 固定步长，不适合刚性 ODE；
- SVD 版本更慢，按需使用。

5.6 与主项目的边界

数值库是通用核心，不读取 FITS/不包含望远镜参数；主项目的正演/反演仍用 CUDA 层。如果主项目需要使用本库的 LU/FFT，应通过明确接口复用，而不是复制实现。

附录 A 关键源码

代码 A-1　LU 与 FFT 的实现骨架。

```
Vector solve_linear(const Matrix& A, const Vector& b) {
  if (A.rows() != A.cols()) throw std::invalid_argument("not square");
  Eigen::FullPivLU<Matrix> lu(A);
  if (!lu.isInvertible()) throw std::runtime_error("singular");
  return lu.solve(b);
}

void fft(std::vector<Complex>& a, bool inverse) {
  if (inverse) for (auto& z : a) z = std::conj(z);
  fft_radix2(a);                       // bit reversal + butterflies
  if (inverse) { for (auto& z : a) z = std::conj(z) / a.size(); }
}
```